

Newton's Method for Several Variables

Numerical solution of nonlinear systems

Sebastián Vargas

Universidad de Pamplona

- 1 Review: Newton's method in one variable
- 2 From one to several variables
- 3 The Jacobian matrix
- 4 Derivation of the method
- 5 Physical example: two coupled masses
- 6 Implementation
- 7 Summary

Reminder: Newton's method in one variable

We want to solve $f(x) = 0$. Starting from an approximation x_k , we approximate f by its tangent line (first-order Taylor expansion):

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k).$$

We set that line equal to zero and solve for the next point x_{k+1} :

$$f(x_k) + f'(x_k)(x_{k+1} - x_k) = 0 \quad \Longrightarrow \quad \boxed{x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}}$$

Core idea: at each step we replace the nonlinear problem by a *linear* one, easy to solve, valid near x_k .

What changes with several variables?

Now we look for a vector $\mathbf{x} = (x_1, \dots, x_n)$ that **simultaneously** makes n functions vanish:

$$\mathbf{F}(\mathbf{x}) = \mathbf{0} \quad \text{with} \quad \mathbf{F}(\mathbf{x}) = \begin{pmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \\ \vdots \\ f_n(\mathbf{x}) \end{pmatrix}.$$

Why a system and not a single equation? With n unknowns, a single equation $f(\mathbf{x}) = 0$ generally leaves a whole set of solutions (a curve, a surface...), not an isolated point. We need *as many equations as unknowns* to pin down a single point.

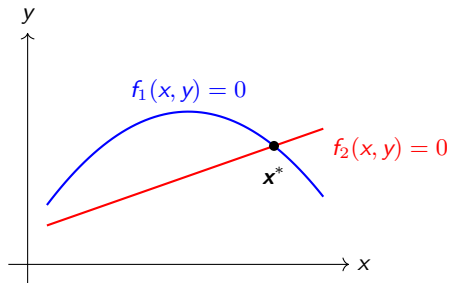
We look for $\mathbf{x}^*$ such that $f_1(\mathbf{x}^*) = f_2(\mathbf{x}^*) = \dots = f_n(\mathbf{x}^*) = 0$.

Geometric interpretation (case $n = 2$)

With two unknowns (x, y) and two functions:

$$\mathbf{F}(x, y) = \begin{pmatrix} f_1(x, y) \\ f_2(x, y) \end{pmatrix}, \quad \text{solving } \mathbf{F} = \mathbf{0} \iff \begin{cases} f_1(x, y) = 0, \\ f_2(x, y) = 0. \end{cases}$$

Each equation $f_i(x, y) = 0$ describes a **curve** in the plane. The solution of the system is the **point where both curves cross**.



Newton's method for several variables looks for that intersection.

What replaces the derivative $f'(x)$?

In one variable, $f'(x)$ tells us how much f changes when we move x .

Now each function f_i can change differently depending on *which* variable we move. We need the derivative of **each function** with respect to **each variable**:

$$\frac{\partial f_i}{\partial x_j} = \text{how much } f_i \text{ changes if we move only } x_j.$$

We organize all these partial derivatives into an $n \times n$ matrix: the **Jacobian matrix** $J(\mathbf{x})$. It is the generalization of $f'(x)$.

$$J(\mathbf{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \cdots & \frac{\partial f_n}{\partial x_n} \end{pmatrix}$$

- **Row** i collects the derivatives of f_i with respect to all the variables.
- **Column** j shows how *every* f_i responds to moving x_j .
- If $n = 1$, $J(x) = f'(x)$: we recover the familiar case.

Taylor linearization (several variables)

Let $\mathbf{x}_k$ be an approximation to the solution and $\Delta \mathbf{x}$ a small step. The vector analogue of $f(x_k) + f'(x_k)(x - x_k)$ is

$$\mathbf{F}(\mathbf{x}_k + \Delta \mathbf{x}) \approx \mathbf{F}(\mathbf{x}_k) + J(\mathbf{x}_k) \Delta \mathbf{x}$$

We want the next point to be (approximately) a zero: $\mathbf{F}(\mathbf{x}_k + \Delta \mathbf{x}) = \mathbf{0}$. Substituting,

$$\mathbf{F}(\mathbf{x}_k) + J(\mathbf{x}_k) \Delta \mathbf{x} \approx \mathbf{0} \quad \implies \quad J(\mathbf{x}_k) \Delta \mathbf{x} = -\mathbf{F}(\mathbf{x}_k)$$

This is an $n \times n$ **linear system** for the unknown $\Delta \mathbf{x}_k$ (we cannot solve for $\Delta \mathbf{x}$ directly as in 1D, since dividing by a matrix makes no sense).

Once $J(\mathbf{x}_k)\Delta\mathbf{x}_k = -\mathbf{F}(\mathbf{x}_k)$ has been solved, we update

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta\mathbf{x}_k$$

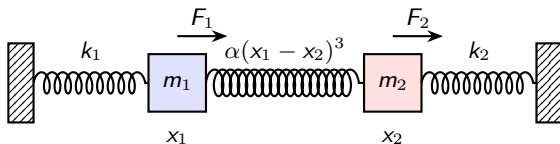
Each iteration consists of four steps:

- 1 Evaluate $\mathbf{F}(\mathbf{x}_k)$.
- 2 Evaluate the Jacobian $J(\mathbf{x}_k)$.
- 3 Solve the linear system $J(\mathbf{x}_k)\Delta\mathbf{x}_k = -\mathbf{F}(\mathbf{x}_k)$.
- 4 Update $\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta\mathbf{x}_k$.

Why not compute J^{-1} ? Formally $\mathbf{x}_{k+1} = \mathbf{x}_k - J(\mathbf{x}_k)^{-1}\mathbf{F}(\mathbf{x}_k)$, but inverting a matrix is expensive and unnecessary: it is much cheaper to *solve* the linear system (Gauss, LU) than to compute the full inverse.

The physical system

Two masses, each attached to a wall by a **linear** spring, and connected to **each other** by a **nonlinear** spring (stiffer the more it stretches). A constant external force F_1 , F_2 also acts on each mass.



x_1, x_2 : displacements of each mass from its rest position. The middle spring is **nonlinear**: its force depends on the *cube* of the relative stretch, $\alpha(x_1 - x_2)^3$ — that is why the problem stops being linear.

Equilibrium equations

At equilibrium, the sum of forces on each mass is zero.

Mass 1: receives F_1 , the wall spring pulls with $-k_1x_1$, and the nonlinear spring pulls with $-\alpha(x_1 - x_2)^3$:

$$F_1 - k_1x_1 - \alpha(x_1 - x_2)^3 = 0.$$

Mass 2: receives F_2 , the wall spring pulls with $-k_2x_2$, and the nonlinear spring pushes with $+\alpha(x_1 - x_2)^3$ (reaction, opposite direction):

$$F_2 - k_2x_2 + \alpha(x_1 - x_2)^3 = 0.$$

With the values $k_1 = 10$, $k_2 = 8$, $\alpha = 20$, $F_1 = 5$, $F_2 = 3$:

$$10x_1 + 20(x_1 - x_2)^3 - 5 = 0$$

$$8x_2 - 20(x_1 - x_2)^3 - 3 = 0$$

Two nonlinear equations, two unknowns: exactly the problem $\mathbf{F}(\mathbf{x}) = \mathbf{0}$ that we know how to solve with Newton's method.

$$\mathbf{F}(x_1, x_2) = \begin{pmatrix} 10x_1 + 20(x_1 - x_2)^3 - 5 \\ 8x_2 - 20(x_1 - x_2)^3 - 3 \end{pmatrix}$$

Differentiating each f_i with respect to x_1 and x_2 (using the chain rule on $(x_1 - x_2)^3$):

$$J(x_1, x_2) = \begin{pmatrix} 10 + 60(x_1 - x_2)^2 & -60(x_1 - x_2)^2 \\ -60(x_1 - x_2)^2 & 8 + 60(x_1 - x_2)^2 \end{pmatrix}$$

At each iteration we solve the 2×2 linear system:

$$J(x_1, x_2) \begin{pmatrix} \Delta x_1 \\ \Delta x_2 \end{pmatrix} = -\mathbf{F}(x_1, x_2).$$

Starting from $\mathbf{x}_0 = (0.2, 0.3)$, Newton's method converges as follows:

Iteration	x_1	x_2	$\ \mathbf{F}(\mathbf{x})\ $
1	0.48986784	0.38766520	1.13×10^{-1}
2	0.49687659	0.37890427	2.27×10^{-3}
3	0.49674161	0.37907299	9.23×10^{-7}
4	0.49674155	0.37907306	1.53×10^{-13}

$$x_1 \approx 0.49674155, \quad x_2 \approx 0.37907306.$$

Notice the **quadratic convergence**: the number of correct digits roughly doubles at each iteration (typical of Newton's method near the solution).

Implementation in Python

```
1 def newton_multivariable(F, J, x0, tol=1e-10, max_iter=50):
2     x = np.array(x0, dtype=float)
3
4     for k in range(max_iter):
5         Fx = F(x)
6         Jx = J(x)
7
8         # Solve J(x_k) Delta x = -F(x_k) (without inverting J)
9         delta = np.linalg.solve(Jx, -Fx)
10
11         x = x + delta
12
13         if np.linalg.norm(F(x)) < tol:
14             return x, k + 1
15
16         raise RuntimeError("Newton did not converge")
17
```

`np.linalg.solve` solves the linear system directly (via LU with pivoting), avoiding the computation of J^{-1} .

Choose $\mathbf{x}_0$. For $k = 0, 1, 2, \dots$ repeat:

$$\boxed{J(\mathbf{x}_k)\Delta\mathbf{x}_k = -\mathbf{F}(\mathbf{x}_k)} \quad \implies \quad \boxed{\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta\mathbf{x}_k}$$

until $\|\mathbf{F}(\mathbf{x}_k)\| < \varepsilon$.

nonlinear problem $\longrightarrow$ local linear problem $\longrightarrow$ repeat until zero

Newton's method for several variables generalizes $f(x) = 0$ to $\mathbf{F}(\mathbf{x}) = \mathbf{0}$:

$$f'(x) \longrightarrow J(\mathbf{x}), \quad x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \longrightarrow \begin{cases} J(\mathbf{x}_k) \Delta \mathbf{x}_k = -\mathbf{F}(\mathbf{x}_k) \\ \mathbf{x}_{k+1} = \mathbf{x}_k + \Delta \mathbf{x}_k \end{cases}$$

Multivariable Newton's method = **analysis** (Taylor) + **linear algebra** (solving systems) + **numerical computation** (iterate until convergence).